

Analisi distribuita di co-occorrenza di terremoti

Corso: Scalable and Cloud Programming – A.A. 2025/2026

Studente: Rosario Zaccone

Repository: github.com/rosario-zaccone/quake-cooccurrence

Obiettivo

L'obiettivo del progetto è di individuare, su un dataset relativo a terremoti in specifiche località e timestamp, la coppia di località distinte che presenta il massimo numero di co-occorrenze giornaliere. Ogni località è individuata da longitudine e latitudine, e come requisito viene richiesta la loro normalizzazione approssimandole alla prima cifra decimale. Viene richiesto inoltre di considerare solo la data nel timestamp. L'output finale è composto dalla coppia di coordinate più frequente e dalla lista ordinata delle date in cui le due località co-occorrono.

Lo stack tecnologico usato è composto da Scala, Apache Spark, Python e Google Cloud.

Implementazione proposta

L'algoritmo generale è il seguente:

1. lettura del dataset da `gs://<bucket>/dataset.csv`;
2. normalizzazione di data e coordinate;
3. raggruppamento degli eventi per data e rimozione dei duplicati;
4. generazione di tutte le coppie distinte di località presenti nello stesso giorno;
5. aggregazione per coppia di località;
6. selezione della coppia più frequente;
7. ordinamento crescente delle date relative alla coppia più frequente.

Vengono proposte cinque varianti, che variano nell'implementare l'algoritmo principale (da passo 3 fino alla fine, poiché il cleaning dei dati è identico in ogni versione), per confrontare diverse metodologie e capire quale soluzione scali meglio.

Preparazione dei dati

Il dataset è fornito sottoforma di CSV, risiede su Google Cloud Storage. Come primo passo viene convertito in un RDD di coppie:

```
(date, Coordinate(longitude, latitude))
```

La data viene estratta dai primi 10 caratteri del campo `date`, mentre le coordinate vengono arrotondate alla prima cifra decimale. Il dataset viene ripartizionato con:

```
repartition(sc.defaultParallelism * partitionMultiplier)
```

Il parametro `partitionMultiplier` consente di confrontare configurazioni con partizionamento pari a 2x, 3x e 4x rispetto al parallelismo di default del cluster.

Solver1

Il primo approccio costituisce la base di partenza, sulla quale sono state fondate le altre versioni di implementazione dell'algoritmo che mirano a migliorarlo. Il raggruppamento per data (passo 3) è realizzato tramite `groupByKey()`, seguito da una conversione in `Set` per rimuovere in modo elegante i duplicati.

Le coppie di località (passo 4) sono enumerate con la rispettiva data tramite una `for-comprehension` con doppio indice, mentre l'aggregazione per coppia (passo 5) usa `groupByKey`: entrambe le operazioni non eseguono alcuna aggregazione parziale, rendendo questa variante potenzialmente la più onerosa. La selezione della coppia più frequente (passo 6) è affidata a `reduce`, che propaga il confronto tra coppie adiacenti attraverso le partizioni fino a restituire il risultato globale.

Solver2

Il secondo approccio introduce la prima ottimizzazione significativa: il raggruppamento per data (passo 3) viene sostituito da `aggregateByKey`, che esegue la deduplicazione delle coordinate già a livello di partizione locale,

riducendo sensibilmente il volume di dati trasferiti durante lo shuffle. Il resto della pipeline rimane invariato rispetto a **Solver1**: la generazione delle coppie usa la stessa for-comprehension con indici, e l'aggregazione per coppia è ancora basata su `groupByKey`.

Solver3

Il terzo approccio mantiene `aggregateByKey` per il passo 3 e introduce una variante per la generazione delle coppie (passo 4): al posto della for-comprehension con doppio indice, viene utilizzato il metodo `.combinations(2)` della libreria standard di Scala. Questa soluzione è più leggibile e dichiarativa, ma genera oggetti intermedi aggiuntivi (`Seq` per ogni coppia), il che può comportare un overhead di allocazione rispetto all'approccio con indici espliciti su dataset di grandi dimensioni. L'aggregazione per coppia rimane basata su `groupByKey`, analogamente a **Solver1** e **Solver2**.

Solver4

Il quarto approccio cerca di ottimizzare sia la generazione delle coppie (passo 4) sia l'aggregazione per coppia (passo 5). La generazione è riscritta con un doppio ciclo `while` esplicito che popola un `ArrayBuffer` mutabile: si evita così l'allocazione di oggetti intermedi tipica delle for-comprehension e di `.combinations`. L'aggregazione per coppia usa `aggregateByKey`, che deduplica le date già durante la fase di combine locale.

Solver5

La generazione delle coppie (passo 4) riprende il doppio ciclo `while` con `ArrayBuffer` di **Solver4**, ma viene eseguita all'interno di `mapPartitions`: anziché invocare una funzione per ogni elemento dell'`RDD`, l'elaborazione avviene per intera partizione. L'aggregazione per coppia usa `aggregateByKey` come in **Solver4**. La riduzione finale (passo 6) sostituisce `reduce` con `treeReduce`, che organizza la riduzione in un albero bilanciato distribuito tra gli esecutori anziché convogliare tutti i risultati parziali verso un unico nodo: questo riduce il costo della riduzione finale organizzandola gerarchicamente.

Configurazione Cloud

Le prove sono state eseguite su Dataproc con la seguente configurazione:

Parametro	Valore
Regione	europa-west1
Tipo macchina	n2-standard-4
Worker testati	2, 3, 4
Partizionamento	2x, 3x, 4x rispetto a <code>sc.defaultParallelism</code>

Gli script in `scripts/` automatizzano creazione dei bucket, upload del dataset, creazione del cluster, sottomissione dei job, raccolta dei log e generazione dei grafici. Le metriche aggregate sono salvate in `results/all_results.csv`, le metriche di una singola esecuzione sono in `results/results.csv`.

Risultati

La Tabella 2 riporta il tempo medio in minuti e secondi dell'esecuzione dei diversi solver, al variare di numero di worker e `partition multiplier`. Si parla di tempo medio poiché sono state eseguite diverse run, per ridurre l'influenza di eventuali variazioni occasionali dovute al carico del sistema, alla schedulazione dei thread o ad altri fattori esterni.

Table 2: Tempi medi di esecuzione dei solver

Solver	Worker	p2x	p3x	p4x
Solver1	2	9 min 36 sec	9 min 13 sec	5 min 40 sec
Solver1	3	10 min 40 sec	6 min 44 sec	5 min 18 sec
Solver1	4	9 min 48 sec	6 min 54 sec	4 min 19 sec
Solver2	2	9 min 53 sec	9 min 15 sec	5 min 50 sec
Solver2	3	10 min 49 sec	6 min 45 sec	5 min 7 sec
Solver2	4	10 min 32 sec	6 min 58 sec	3 min 56 sec
Solver3	2	11 min 38 sec	10 min 52 sec	7 min 3 sec
Solver3	3	12 min 12 sec	7 min 58 sec	6 min 35 sec
Solver3	4	11 min 57 sec	7 min 57 sec	5 min 23 sec
Solver4	2	27 min 27 sec	21 min 53 sec	17 min 42 sec
Solver4	3	25 min 10 sec	17 min 10 sec	13 min 48 sec
Solver4	4	22 min 3 sec	15 min 51 sec	10 min 27 sec
Solver5	2	24 min 52 sec	25 min 27 sec	17 min 30 sec
Solver5	3	24 min 32 sec	17 min 16 sec	15 min 15 sec

Table 2: Tempi medi di esecuzione dei solver

Solver	Worker	p2x	p3x	p4x
Solver5	4	20 min 56 sec	15 min 18 sec	12 min 12 sec

La coppia di coordinate con il maggior numero di co-occorrenze è composta dai punti (lon=-122.8, lat=38.8) e (lon=-122.7, lat=38.8)

Analisi prestazioni

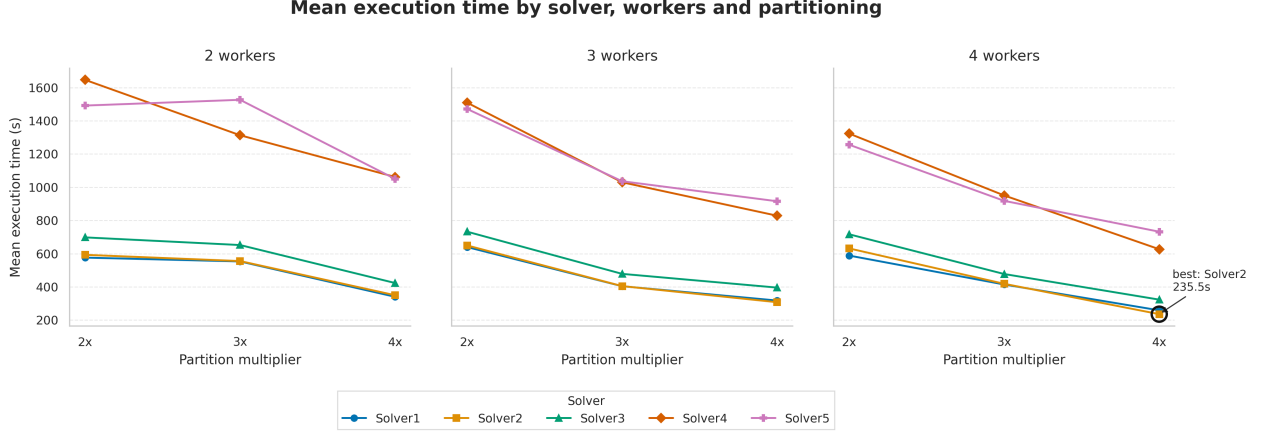


Figure 1: Grafico prestazioni

Dalla Figura 1 si osservano dei tempi di esecuzioni molto diversi. Si osserva che **Solver1** e **Solver2**, su questo dataset, risultano più performanti rispetto agli altri algoritmi.

Solver4 e **Solver5** presentano i tempi di esecuzione peggiori. Si distinguono dagli altri algoritmi per l'uso di **aggregateByKey** nella fase di raggruppamento delle coppie: invece di raccogliere i valori grezzi tramite **groupByKey**, costruiscono il **Set** di date già localmente su ogni nodo, riducendo il volume dei dati trasferiti attraverso la rete. Sul dataset attuale questo overhead non ripaga, poiché supera il risparmio ottenuto dallo shuffle ridotto. Al crescere del dataset però, si ha un numero di coppie maggiore e in questo caso il **groupByKey** dei primi solver diventerebbe un collo di bottiglia, mentre l'approccio di **Solver4** e **Solver5** si rivelerebbe vincente, come suggerito anche dal loro speedup superiore.

Osservando invece come varia il tempo di esecuzione in base al **partition multiplier**, si nota che passando da 2x a 3x e soprattutto a 4x, i tempi di esecuzione tendono a diminuire, effetto che si nota soprattutto nei solver più lenti. Questo perché aumentando il numero di partizioni andiamo a suddividere il dataset in chunk più piccoli, distribuendo meglio il carico di lavoro. Per **Solver4** e **Solver5**, che eseguono computazioni costose localmente, avere partizioni di piccolo volume significa elaborare meno coppie per task, e un parallelismo più efficace.

L'aumento del numero di worker non produce sempre un miglioramento uniforme, si nota soprattutto nel caso con **partition multiplier 2x**. Possiamo dedurre che con una granularità di partizionamento molto bassa il carico non viene distribuito in modo efficace, oppure il lavoro di coordinamento dei worker diventa rilevante rispetto al lavoro utile svolto. Nei casi di **partition multiplier 3x** e **4x** possiamo vedere come l'aumento dei worker risulti in un tempo di esecuzione migliore, probabilmente dato da un buon bilanciamento fra worker disponibili e partizioni. La configurazione migliore si osserva per **Solver2** con **partition multiplier 4x** e 4 worker, che raggiunge il tempo medio più basso. Questo conferma che i solver più efficienti beneficiano maggiormente di una combinazione tra maggiore parallelismo e maggiore granularità del partizionamento.

Analisi scalabilità

Per misurare la scalabilità vengono esaminate tre metriche: speedup, strong scaling efficiency e weak scaling efficiency.

Speedup

Lo **speedup** è stato calcolato usando come baseline la configurazione con 2 worker ed è definito come:

$$S(w) = \frac{T_2}{T_w}$$

dove T_2 è il tempo medio di esecuzione ottenuto con 2 worker, mentre T_w è il tempo medio di esecuzione ottenuto con w worker.

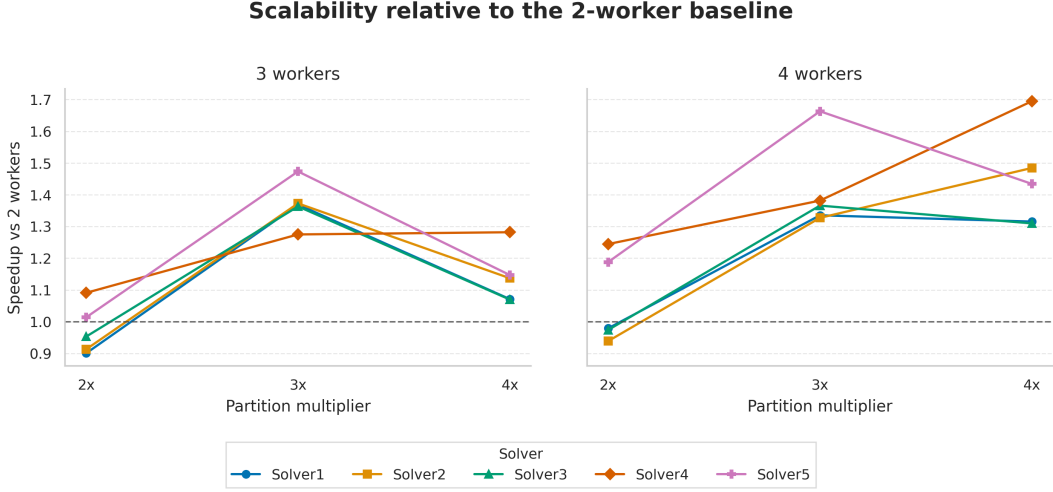


Figure 2: Grafico speedup

Nella Figura 2 è riportato lo *speedup* ottenuto dai diversi solver rispetto alla baseline a 2 worker, al variare del numero di worker e del *partition multiplier*. Con 3 worker lo speedup risulta moderato, con un andamento piuttosto uniforme tra i solver. Il valore migliore si osserva in corrispondenza di *partition multiplier* pari a 3x, dove quasi tutti i solver raggiungono il proprio picco, in particolare **Solver5** che si avvicina molto allo speedup ideale 1.5. Con *partition multiplier* 4x si ha un calo dello speedup nella maggiorparte dei casi, ciò indica che un aumento eccessivo delle partizioni può introdurre overhead di scheduling e shuffle, riducendo il beneficio del parallelismo. Fa eccezione **Solver4**, il cui speedup non cala con *partition multiplier* 4x.

Con 4 worker lo speedup diventa più evidente. **Solver4** e **Solver5** sono i solver che si avvicinano di più allo speedup ideale di 2. **Solver5** raggiunge il suo picco con *partition multiplier* 3x, per poi calare, mentre **Solver4** ha un aumento dello speedup uniforme con l'aumento delle partizioni, che raggiunge il suo picco con *partition multiplier* 4x.

Si può supporre che la migliore scalabilità di **Solver4** e **Solver5** sia correlata alla riduzione dello shuffle fra nodi, poichè a differenza dei primi tre algoritmi evitano un secondo `groupByKey` e aggregano direttamente le date tramite `aggregateByKey`. Inoltre, **Solver4** genera le coppie con cicli espliciti, mentre **Solver5** sfrutta anche `mapPartitions` e `treeReduce`, rendendo più efficiente la distribuzione del carico.

Strong Scaling Efficiency

La **strong scaling efficiency** misura l'efficienza dello speedup mantenendo fissa la dimensione del problema. Poiché la baseline è a 2 worker, è definita come:

$$E(w) = \frac{S(w)}{\frac{w}{2}}$$

dove $\frac{w}{2}$ rappresenta lo speedup ideale rispetto alla configurazione base a 2 worker. Il valore ideale è 1.

Nella Figura 3 è riportata la *strong scaling efficiency* dei diversi solver rispetto alla baseline a 2 worker, al variare del numero di worker e del *partition multiplier*.

Con 3 worker, l'efficienza risulta ottima, si osservano i valori migliori con *partition multiplier* pari a 3x, ad eccezione di **Solver4**. Con 4x, invece, si osserva una riduzione dell'efficienza, segnale che l'aumento delle partizioni introduce un overhead che limita il guadagno ottenuto dal parallelismo.

Con 4 worker l'efficienza è mediamente più bassa, poichè aumentano i costi di coordinamento e comunicazione, che non vengono compensati dal corrispondente aumento dello speedup. Tuttavia, **Solver4** mantiene il comportamento più stabile e raggiunge il valore migliore con *partition multiplier* 4x, confermando una migliore capacità di sfruttare le risorse disponibili rispetto agli altri solver.

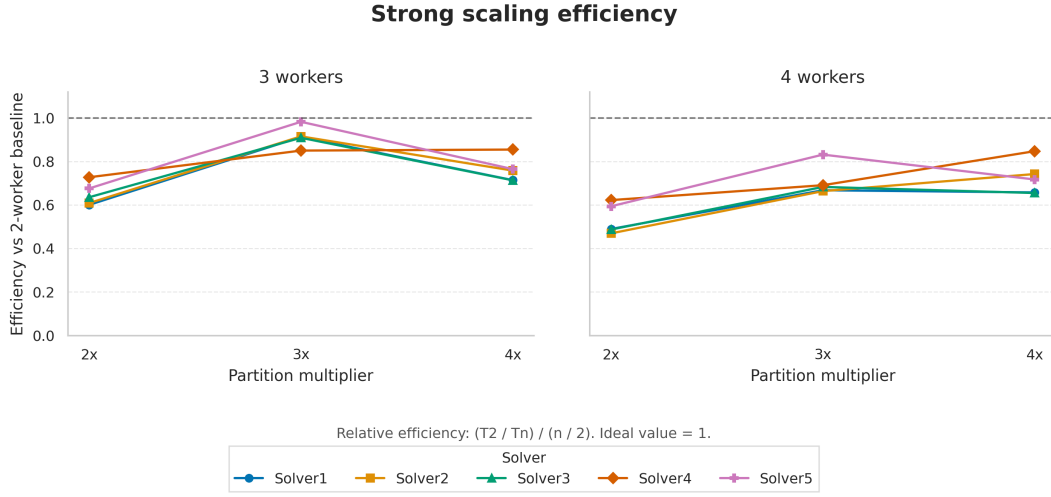


Figure 3: Grafico strong scaling efficiency

Weak Scaling Efficiency

La **weak scaling efficiency** misura come varia il tempo di esecuzione se il problema cresce e si aggiungono dei nodi di calcolo, ipotizzando che la quantità di lavoro svolta da ogni nodo rimanga fissa. La metrica è definita come:

$$W(n) = \frac{T_2}{T_n}$$

dove T_2 è il tempo medio per completare il workload di riferimento con 2 worker, mentre T_n è il tempo medio ottenuto con n worker e workload scalato di un fattore $n/2$. Nella Figura 4 è illustrato l'andamento della metrica al variare dei solver e del partition multiplier, usando la configurazione a 2 worker come baseline ed esaminando la configurazione a 4 worker. Per il calcolo della weak scaling, il dataset è stato scalato replicando i record originali di un fattore pari a `workers / 2`; le repliche successive alla prima sono state rese distinte modificando la data e traslando le coordinate, così da aumentare il workload senza introdurre duplicati identici. Anche qui si nota la superiorità di **Solver4** e **Solver5** in termini di scalabilità.

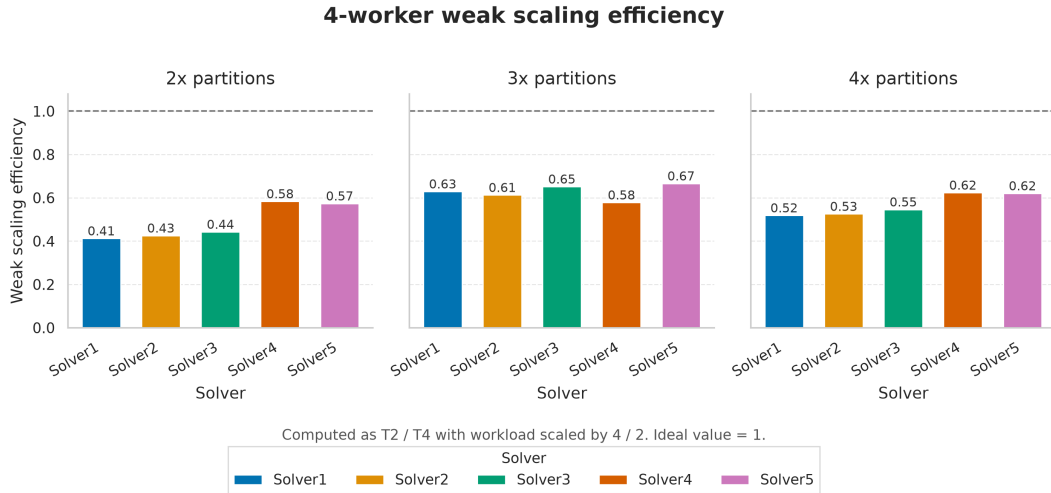


Figure 4: Grafico weak scaling efficiency

In conclusione, **Solver4** e **Solver5** non sono i più veloci sul dataset analizzato, ma mostrano una migliore capacità di scalare. Questo dipende soprattutto dall'uso di `aggregateByKey`, che riduce lo shuffle rispetto a `groupByKey`. Inoltre, la generazione delle coppie con `while` e `ArrayBuffer` rende il calcolo più controllato. **Solver5** aggiunge anche `mapPartitions` e `treeReduce`, migliorando la gestione del lavoro tra le partizioni. Per questo motivo, questi solver risultano più promettenti in scenari con dataset più grandi o con più risorse disponibili.